

~~BFS~~ ~~Breadth first search~~.

~~Ass~~ Assumptions:

- Graph representation: Adjacency list (most common)
- graph type: general graph (can be directed/undirected)
- Number of vertices (nodes) = N
- Number of Edges = E
- for BFS & DFS we assume we want to visit all reachable values.

BFS:- (Breadth first search)

- BFS explores level by level (neighbours first, then neighbours' neighbours)
- It uses QUEUE to store the frontier (vertices whose neighbours have not been fully explored yet).
- Typical implementation:
 - mark source as visited, enqueue it.
 - while queue not empty $\rightarrow$ dequeue a node, explore its adjacency list, enqueue unvisited neighbours
- Time complexity:
 - Visiting every vertex take $O(N)$.
 - for each vertex, we scan Adjacency list = $O(E)$
 - Queue operations (enqueue/dequeue) are $O(1)$ each, done once per vertex = $O(N)$

$$\text{Total Time} = O(N+E)$$

Space complexity (BFS)

- visited array: ~~$O(N)$~~ $O(N)$
- Queue: upto $O(N)$ in (all vertices) worst case
- Adjacency list storage: $O(N+E)$ (Built in graph representation)

Total space complexity = $O(N+E)$

DFS (depth first search)

- DFS explores deep before wide (go as far down a branch before backtracking)
- Can be implemented via ~~Explicit~~ stack (data structure)
- ~~Recursion~~
- Each vertex visiting $\rightarrow O(N)$
for each vertex, we traverse its adjacency list once $\rightarrow$ ~~$O(E)$~~ $O(E)$
- Stack push/pop or recursive call $\rightarrow O(1)$
- Time complexity = $O(N+E)$

Space complexity (DFS)

- array visiting: $O(N)$
- stack in worst case = $O(N)$
- Adjacency - $O(N+E)$
- Total space = $O(N+E)$

Sparse graph

- $E \approx O(N)$
- complexity - $O(N)$
- linear no. of nodes.

Dense graph

- $E \approx O(N^2)$
- complexity $O(N^2)$